

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\agent-af2b29f0bbdf75b59.jsonl`  
- SHA-256: `ab0998d1230ec33f8eb174ac97c1e1d4e30d3ed167a10f77e2562ea27141c2d4`  
- Source modified: `2026-06-21T15:30:23+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `subagents`  
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

Transcript

1. user (2026-06-21T15:27:01.579Z)

Adversarially review a network-exposure hardening PR (PACKAGING_PLAN.md P0c) in the git worktree C:/Users/avidu/Projects/bhi-wt-p0c. It (1) adds a profile-aware bind-host resolver and a --host arg, and (2) adds a Starlette TrustedHostMiddleware DNS-rebinding guard with an env-driven allowlist.

Read in that worktree:

```
- backend/main.py ? the new `_allowed_hosts_from_env`, the `TrustedHostMiddleware`  
add_middleware call, `_resolve_bind_host`, the `--host` arg, and the server-start block (search  
"TrustedHost", "_resolve_bind_host", "bind_host", "uvicorn.run").  
- tests/test_bind_and_host.py  
- backend/config/models.py SecurityConfig (edge_auth_enabled).
```

HARD CONTRACT: default (edge_auth_enabled=False, no --host/env) must be byte-identical to before ? bind 127.0.0.1, and the full existing TestClient suite (Host header "testserver") must not break.

Adversarially check, with file:line evidence:

1. Does the global TrustedHostMiddleware break any LEGITIMATE access? Confirm "testserver" (Starlette TestClient default Host) is allowlisted so the suite passes. Does it correctly allow a loopback Host WITH a port (e.g. "127.0.0.1:8000")? Does Starlette strip the port before matching? Any path (health, static "/", CORS preflight OPTIONS) that the middleware would wrongly 400?
2. The edge-auth interaction: when edge_auth_enabled=True, bind becomes 0.0.0.0 but the middleware allowlist is resolved at IMPORT from env only (it can't see config). If RALLY_ALLOWED_HOSTS is unset, the loopback-only default would 400 all forwarded (public-Host) requests. Is this footgun adequately surfaced (the bind-time warning) and documented? Is there a cleaner option (e.g., allowlist also reading a profile/env signal)? Is the precedence in _resolve_bind_host (cli > env > profile > 127.0.0.1) correct and safe?
3. Middleware ORDER: TrustedHost is added after CORS ? does that cause a CORS preflight (OPTIONS with Origin) from a disallowed Host to behave oddly, or leak CORS headers before the host check? Is www_redirect=False correct?
4. Any byte-identity break for the default path (e.g., the server-start print changed from "localhost" to the bind host; the --host default None)? Is `getattr(sec, "edge\_auth\_enabled", False)` safe if config or security is None?
5. IPv6: is "[::1]"/"::1" handled (Host header for IPv6 is "[::1]:port")? Minor, but note it.

Return ONLY: findings as "[SEVERITY blocker|major|minor|nit] <file:line> ? <issue> ? <fix>", then a one-line verdict. Be precise; don't invent issues. If clean, say so.

2. assistant (2026-06-21T15:27:09.599Z)

I'll review this network-exposure hardening PR. Let me start by reading the relevant files in the worktree.

3. user (2026-06-21T15:27:11.401Z)

```
fc78fce feat(pkg): P0b ? redact secret-shaped fields from the unauthenticated GET /api/config (#295)
0c0e064 feat(serving): exposure-safety boot posture ? fail fast on a half-configured exposed engine (default-OFF) (#294)
82a28f4 feat(pkg): P0a ? resolved app-data HOME (RALLY_APP_HOME) for config/.env/DB/output (default-OFF) (#293)
2afbadd fix(dryrun): install the watermark font in bootstrap + track the UI-trigger asks (#292)
7835f34 feat(gcp): always-install Ops Agent on GPU VMs + a cloud-GPU dry-run guide (#291)
---STATUS---
On branch feat/packaging-p0c-bind-host
Your branch is up to date with 'origin/master'.
```

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   backend/main.py
```

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    tests/test_bind_and_host.py
```

```
no changes added to commit (use "git add" and/or "git commit -a")
```

4. user (2026-06-21T15:27:11.853Z)

```
1      """Bind-address resolution + DNS-rebinding Host-header guard (PACKAGING_PLAN.md P0c)."""
2
3      import pytest
4
5      pytest.importorskip("httpx") # TestClient needs httpx
6      from fastapi.testclient import TestClient
7
8      import backend.main as m
9      from backend.config import AppConfig
10
11
12      @pytest.fixture(autouse=True)
13      def _clear_env(monkeypatch):
14          for var in ("RALLY_BIND_HOST", "RALLY_ALLOWED_HOSTS"):
15              monkeypatch.delenv(var, raising=False)
16
17
18      # ---- bind host resolution ----
19
20      def test_bind_defaults_to_localhost():
21          assert m._resolve_bind_host(AppConfig(), None) == "127.0.0.1"
22
23
24      def test_bind_cli_host_wins(monkeypatch):
25          monkeypatch.setenv("RALLY_BIND_HOST", "10.0.0.5")
26          cfg = AppConfig()
27          cfg.security.edge_auth_enabled = True
28          assert m._resolve_bind_host(cfg, "192.168.1.9") == "192.168.1.9" # --host beats env
+ profile
29
30
31      def test_bind_env_beats_profile(monkeypatch):
32          monkeypatch.setenv("RALLY_BIND_HOST", "0.0.0.0")
33          assert m._resolve_bind_host(AppConfig(), None) == "0.0.0.0"
34
35
36      def test_bind_edge_auth_profile_widens_to_all_interfaces():
37          cfg = AppConfig()
38          cfg.security.edge_auth_enabled = True
39          assert m._resolve_bind_host(cfg, None) == "0.0.0.0"
```

```

40
41
42     def test_bind_no_config_is_localhost():
43         assert m._resolve_bind_host(None, None) == "127.0.0.1"
44
45
46     # ---- allowed-hosts resolution ----
47
48     def test_allowed_hosts_default_is_loopback_plus_testserver():
49         hosts = m._allowed_hosts_from_env({})
50         assert "127.0.0.1" in hosts and "localhost" in hosts and "testserver" in hosts
51         assert "" not in hosts
52
53
54     def test_allowed_hosts_env_override():
55         assert m._allowed_hosts_from_env({"RALLY_ALLOWED_HOSTS": "app.example.com,
1.2.3.4"}) == [
56             "app.example.com", "1.2.3.4"]
57
58
59     def test_allowed_hosts_wildcard_for_edge():
60         assert m._allowed_hosts_from_env({"RALLY_ALLOWED_HOSTS": "*"}) == ["*"]
61
62
63     # ---- the middleware actually rejects a foreign Host (DNS-rebinding) ----
64
65     def test_foreign_host_header_is_rejected():
66         client = TestClient(m.app)
67         r = client.get("/api/health", headers={"host": "evil.attacker.com"})
68         assert r.status_code == 400 # TrustedHostMiddleware: Invalid host header
69
70
71     def test_loopback_host_header_is_accepted():
72         client = TestClient(m.app)
73         # default TestClient host is "testserver" (allowed); an explicit loopback host (with
port) too.
74         assert client.get("/api/health").status_code == 200
75         assert client.get("/api/health", headers={"host": "127.0.0.1:8000"}).status_code ==
200
76

```

5. user (2026-06-21T15:27:11.995Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
packaged
11    # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
keeps the
12    # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
13    # stdlib-only import ? keeps this pre-pydantic line light.
14    from backend.utils.app_home import env_file_to_load, resolve_output_dir
15
16    load_dotenv(env_file_to_load())
17
18    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).

```

```

19     logging.basicConfig(
20         level=logging.INFO,
21         format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
22         datefmt='%H:%M:%S'
23     )
24     logger = logging.getLogger(__name__)
25     from fastapi import FastAPI, HTTPException, Request
26     from fastapi.middleware.cors import CORSMiddleware
27     from starlette.middleware.trustedhost import TrustedHostMiddleware
28     from pydantic import BaseModel
29     from typing import List, Dict, Any, Optional
30
31     from fastapi.staticfiles import StaticFiles
32     from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
33     from backend.pipeline.validators.base import registry
34     from backend.infrastructure.database import Database
35     from backend.pipeline.segmenters.base import segmenter_registry
36     from backend.pipeline.stitcher.compiler import VideoCompiler
37     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
38     from backend.utils.paths import resolve_under_root, safe_output_filename,
39     validate_input_path
40     from backend.utils.redact import redact_secrets # P0b: mask secret-shaped fields in
41     /api/config
42     from backend.config import ( # new typed config
43         load_config as load_app_config,
44         AppConfig,
45         StitchingConfig,
46         enforce_startup_checks,
47         StartupCheckError,
48     )
49     from backend.results import Failure # standardized error/result contract
50     from backend.reporting import emit_indexer_report
51     from backend import storage # M2: URI-aware input acquisition (local = identity
52     passthrough)
53     from backend import billing # M8: per-job cost ledger (USD internal / INR display)
54     from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-
55     OFF)
56
57     app = FastAPI(title="Sports Highlight Indexer API")
58
59     def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
60         """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
61         separated),
62         default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
63         ``backend.main``
64         does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
65         its origin."""
66         src = os.environ if env is None else env
67         raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
68         origins = [o.strip() for o in raw.split(",") if o.strip()]
69         return origins or ["*"]
70
71     # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by
72     browsers per the
73     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
74     credentials stay
75     # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
76     deploy can
77     # lock it to its origin; the middleware is fixed at startup.
78     app.add_middleware(
79         CORSMiddleware,
80         allow_origins=_cors_origins_from_env(),
81         allow_credentials=False,

```

```

74     allow_methods=["*"],
75     allow_headers=["*"],
76 )
77
78
79 def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
80     """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
malicious
81     web page can point a domain it controls at 127.0.0.1 and script requests to the
local server;
82     the browser sends ``Host: attacker.com``, which this rejects (the server only
answers loopback
83     Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the
default keeps
84     Starlette's TestClient host ``testserver`` so the in-process suite isn't broken.
Read from the
85     ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a
routable/edge
86     deploy sets its public host (or ``*`` when fronted by an auth gate that validates
the host)."""
87     src = os.environ if env is None else env
88     raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
89     hosts = [h.strip() for h in raw.split(",") if h.strip()]
90     return hosts or ["localhost", "127.0.0.1", "::1", ":::1", "testserver"]
91
92
93     # DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are
honoured
94     # (a remote page can't read responses cross-origin AND its Host header is rejected
here). www_redirect
95     # off so an unexpected ``www.`` host is rejected rather than 307-redirected.
96     app.add_middleware(
97         TrustedHostMiddleware,
98         allowed_hosts=_allowed_hosts_from_env(),
99         www_redirect=False,
100     )
101
102
103 def _resolve_bind_host(config: Optional[AppConfig] = None, cli_host: Optional[str] =
None) -> str:
104     """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
``0.0.0.0``
105     when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS
an auth gate
106     in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a
routable
107     interface with no auth gate is unsafe, hence the localhost default; widen it
consciously."""
108     if cli_host and cli_host.strip():
109         return cli_host.strip()
110     env = os.environ.get("RALLY_BIND_HOST")
111     if env and env.strip():
112         return env.strip()
113     sec = getattr(config, "security", None)
114     if getattr(sec, "edge_auth_enabled", False):
115         return "0.0.0.0" # behind a Cloudflare-Access-style edge gate (M9) ? bind all
interfaces
116     return "127.0.0.1"
117
118     # Serves static frontend files directly (now under api/ per approved structure).
119     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
an
120     # arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
121     # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)

```

```

122     _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
123     try:
124         os.makedirs(_STATIC_DIR, exist_ok=True)
125     except OSError:
126         # The dir ships with the package (a no-op create); guard against a read-only install
127         # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
128         # PermissionError.
129         pass
130     app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
131
132     @app.get("/", response_class=HTMLResponse)
133     def serve_index():
134         index_path = os.path.join(_STATIC_DIR, "index.html")
135         if os.path.exists(index_path):
136             with open(index_path, "r", encoding="utf-8") as f:
137                 return f.read()
138         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
139         backend/api/static/</h3>"
140
141     # Global variables initialized at startup
142     db_instance: Optional[Database] = None
143     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
144
145     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
146     # P3)
147     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
148     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
149     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
150     # THROUGH
151     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
152     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
153     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
154     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
155     on backend.main)
156     JobWaiting,
157     _arm_wake_timer,
158     _drain_due_jobs,
159     _get_executor,
160     _job_to_request,
161     _MAX_DRAIN_WAKE_SEC,
162     _maybe_record_job_cost,
163     _run_claimed_job,
164     _schedule_next_drain_if_waiting,
165     )
166
167     # Legacy thin wrapper for any remaining direct calls during transition.
168     # New code should import load_app_config / AppConfig from backend.config directly.
169     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
170         cfg = load_app_config(config_path)
171         return cfg.model_dump(mode="json")
172
173     # --- API Models ---
174     class ProcessRequest(BaseModel):
175         video_path: str
176         player_pool: Optional[List[str]] = None
177         max_frames: Optional[int] = None
178         segmenter: Optional[str] = None
179
180     class QueryRequest(BaseModel):
181         video_id: str
182         server: Optional[str] = None
183         receiver: Optional[str] = None
184         duration_min: Optional[float] = None
185         event_type: Optional[str] = None

```

```

182 class CompileRequest(BaseModel):
183     video_id: str
184     segment_ids: List[int]
185     output_filename: str
186
187 def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
188     """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
189
190     On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
191     fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
192     and
193     keep the prior good results intact ? a failed/empty re-run never destroys prior
194     data.
195     """
196     if segments:
197         db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
198     else:
199         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
200
201 # --- API Endpoints ---
202 @app.get("/api/config")
203 def get_config():
204     # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
205     field. Today
206     # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
207     id), so this
208     # is byte-identical for the real fields ? but it fails-closed if a secret is ever
209     added (guarded
210     # by tests/test_redact.py + the /api/config redaction test).
211     if global_config is None:
212         return {}
213     return redact_secrets(global_config.model_dump(mode="json"))
214
215 @app.get("/api/health")
216 def health():
217     """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
218     (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
219     confirm the engine is reachable and which detector is wired. Never raises."""
220     sport = getattr(global_config, "sport", None)
221     indexing = getattr(global_config, "indexing", None)
222     return {
223         "status": "ok",
224         "sport": sport,
225         "default_segmenter": getattr(indexing, "default_segmenter", None),
226     }
227
228 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
229     """The full hash ? validate ? segment ? report pipeline for one video.
230
231     This is the former synchronous /api/process body, unchanged in behavior, now run on
232     the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
233     the sync endpoint used to return (UI compatibility); the per-video observability
234     report is still emitted in `finally:` and grafted as response["reports"].
235
236     `progress` is an optional callable(stage: str) used to surface coarse job progress.
237     Raises on unexpected internal errors ? the caller records those as a job failure
238     (after this function has already restored the pre-run segments, see below).
239     """
240     notify = progress or (lambda stage: None)
241
242     notify("hashing")
243
244     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
245     (identity ?
246     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The

```

```

original
241     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
242     # consumers (hash / validators / segmenter) use the resolved local path.
243     local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
244     video_id = compute_video_id(local_video)
245     was_reingest = db_instance.get_video(video_id) is not None
246
247     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
248     notify("validating")
249     logger.info(f"Running validations for {req.video_path}...")
250     val_results, val_context = registry.run_all_with_context(local_video, global_config)
251     failures = [r for r in val_results if not r.passed]
252
253     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
254     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
255     seg_name = req.segmenter or default_seg
256     segmenter_actual = None
257     segmentation_ran = False
258     response: Optional[Dict[str, Any]] = None
259     try:
260         if failures:
261             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
262             # status; it no longer destroys the video's prior good segments.
263             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
264             response = {
265                 "video_id": video_id,
266                 "status": "failed",
267                 "message": "Video failed validation checks.",
268                 "results": [r.model_dump() for r in val_results],
269             }
270             return response
271
272     # 2. Run segmenter & indexer
273     logger.info("[API] Video passed checks. Segmenting and indexing...")
274     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
275
276     segmenter_cls = segmenter_registry.get(seg_name)
277     if not segmenter_cls:
278         # Submit-time validation already 400s unknown names; this is the defensive
279         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
280         available = list(segmenter_registry.list_available().keys())
281         response = {
282             "video_id": video_id,
283             "status": "failed",
284             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
285             "results": [r.model_dump() for r in val_results],
286             "play_segments": [],
287         }
288         return response
289
290     logger.info(f"[API] Using segmenter: {seg_name}")
291     notify(f"segmenting ({seg_name})")
292     segmenter_actual = seg_name
293     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
294     # if the run yields segments; otherwise drop the new partial rows and keep the
old.

```



```

295         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
296         segmenter = segmenter_cls(db_instance, global_config)
297         try:
298             result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
299         except Exception:
300             # A raising segmenter must not leave half-written rows: restore the pre-run
301             # state (same destroy-after-confirm contract as the Failure branch), then
302             let
303                 # the job worker record the failure.
304                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
305                 raise
306             segmentation_ran = True
307
308         if isinstance(result, Failure):
309             logger.error(f"[API] Segmenter failed: {result.message}")
310             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
311             response = {
312                 "video_id": video_id,
313                 "status": "failed",
314                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
315                 "results": [r.model_dump() for r in val_results],
316                 "play_segments": [],
317             }
318             return response
319
320         segments = result.value if hasattr(result, "value") else result
321         notify("finalizing")
322         _finalize_reingest(video_id, segments, play_wm, cand_wm)
323
324         response = {
325             "video_id": video_id,
326             "status": "success",
327             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
328             "results": [r.model_dump() for r in val_results],
329             "play_segments": segments,
330         }
331         return response
332
333     finally:
334         # Always emit the per-video observability report (next to the video, or to
335         # report.output_dir). Never raises. Surface the paths in the response.
336         paths = emit_indexer_report(
337             db=db_instance, video_id=video_id, video_path=req.video_path,
338             config=global_config,
339             val_results=val_results, val_context=val_context,
340             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
341             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
342         )
343
344         if paths and isinstance(response, dict):
345             response["reports"] = paths
346
347
348     @app.post("/api/process", status_code=202)
349     def process_video(req: ProcessRequest, request: Request):
350         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
351
352         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
353         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
354         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
355         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
356
357         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
358         single-user,
359         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a

```

```

missing or
355         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
356         try:
357             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
358         except edge_auth.EdgeAuthError as e:
359             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
360
361         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
362         try:
363             validate_input_path(req.video_path, allowed_root=allowed_root)
364             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
365             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
366             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
367         except ValueError as e:
368             raise HTTPException(status_code=400, detail=str(e)) from e
369         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
370         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
371         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
372         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
373         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
374         # rejects a non-local URI as out-of-root.
375         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
376             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
377         if req.segmenter and not segmenter_registry.get(req.segmenter):
378             available = list(segmenter_registry.list_available().keys())
379             raise HTTPException(
380                 status_code=400,
381                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}")
382         )
383
384         job_id = uuid.uuid4().hex
385         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
386                                     req.player_pool, req.max_frames, owner_id=owner_id):
387             raise HTTPException(status_code=500, detail="Failed to persist job record.")
388         _get_executor().submit(_drain_due_jobs)
389         return JSONResponse(
390             status_code=202,
391             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
392         )
393
394
395         @app.get("/api/jobs/{job_id}/cost")
396         def get_job_cost(job_id: str):
397             """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
398             GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
399             null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
400             cost = db_instance.get_job_cost(job_id)
401             if cost is None:
402                 raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
403             fx = global_config.billing.fx_inr_per_usd
404             usd = cost.get("cost_usd")
405             cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None

```

```

406         cost["fx_inr_per_usd"] = fx
407         return cost
408
409
410     @app.get("/api/videos/{video_id}/cost")
411     def get_video_cost(video_id: str):
412         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
413         ``total_cost_inr`` + the
414         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
415         several."""
416         agg = db_instance.get_video_cost(video_id)
417         fx = global_config.billing.fx_inr_per_usd
418         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
419         agg["fx_inr_per_usd"] = fx
420         return agg
421
422     @app.get("/api/jobs/{job_id}")
423     def get_job(job_id: str):
424         """Job state: {status, progress, result, reports}. `status` = execution state
425         (queued|running|done|failed); the pipeline verdict is result["status"]."""
426         job = db_instance.get_job(job_id)
427         if not job:
428             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
429         result = job.get("result")
430         return {
431             "job_id": job["id"],
432             "status": job["status"],
433             "progress": job.get("progress"),
434             "video_id": job.get("video_id"),
435             "error": job.get("error"),
436             "result": result,
437             "reports": (result or {}).get("reports"),
438             "created_at": job.get("created_at"),
439             "started_at": job.get("started_at"),
440             "finished_at": job.get("finished_at"),
441         }
442
443     # --- Chunked upload (B2, PR-2) -----
444     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
445     # in
446     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
447     # via
448     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
449     # sequential-part logic, and abort cleanup ? is unchanged.
450     from backend.api import uploads as uploads_api
451
452     app.include_router(uploads_api.router)
453
454     # --- Highlight download (B3, PR-2) -----
455
456     @app.get("/api/highlights/{video_id}/{filename}")
457     def download_highlight(video_id: str, filename: str):
458         """Stream a compiled highlight reel ? `filename` is the bare name given to
459         /api/compile
460         (which only writes into output_dir; the browser previously got back an unreachable
461         server-local path)."""
462         try:
463             name = safe_output_filename(filename)
464         except ValueError as e:
465             raise HTTPException(status_code=400, detail=str(e)) from e
466         if not db_instance.get_video(video_id):
467             raise HTTPException(status_code=404, detail="Indexed video record not found.")
468         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")

```

```

or "output"
466     path = os.path.join(output_dir, name)
467     try:
468         path = resolve_under_root(path, output_dir)
469     except ValueError as e:
470         raise HTTPException(status_code=400, detail=str(e)) from e
471     if not os.path.isfile(path):
472         raise HTTPException(status_code=404,
473                             detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
474     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
475     return FileResponse(path, media_type=media, filename=name)
476
477
478 @app.post("/api/query")
479 def query_play_segments(req: QueryRequest):
480     filters = {
481         "server": req.server,
482         "receiver": req.receiver,
483         "duration_min": req.duration_min,
484         "event_type": req.event_type
485     }
486     segments = db_instance.query_play_segments(req.video_id, filters)
487     return {"play_segments": segments}
488
489 @app.post("/api/compile")
490 def compile_highlights(req: CompileRequest):
491     video = db_instance.get_video(req.video_id)
492     if not video:
493         raise HTTPException(status_code=404, detail="Indexed video record not found.")
494
495     # Retrieve matching play segments from db
496     all_segments = db_instance.query_play_segments(req.video_id, {})
497     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
498
499     if not matched_segments:
500         raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
501
502     # Reject path traversal / absolute output names (os.path.join would otherwise let an
503     # absolute or '..'-laden output_filename write anywhere on disk).
504     try:
505         out_name = safe_output_filename(req.output_filename)
506     except ValueError as e:
507         raise HTTPException(status_code=400, detail=str(e)) from e
508
509     # The configured shot-count floor (stitching.min_shot_count) applies on the API
510     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
511     # metadata are exempt: the floor filters known counts only, so explicitly
512     # requested manual/legacy segments can't silently vanish under the default floor.
513     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
514     kept = []
515     for s in matched_segments:
516         meta = s.get("metadata") or {}
517         sc = meta.get("shot_count") if isinstance(meta, dict) else None
518         try:
519             if sc is not None and int(sc) < stitching.min_shot_count:
520                 continue
521         except (TypeError, ValueError):
522             pass # unparseable count -> treat as unknown, keep
523         kept.append(s)
524     if len(kept) < len(matched_segments):
525         logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped ")

```

```

526             f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
527         if not kept:
528             raise HTTPException(status_code=400,
529                                 detail=f"All {len(matched_segments)} matching segments fall
below "
530
531 f"stitching.min_shot_count={stitching.min_shot_count}.")
532         # Extract start/end time pairs
533         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
534
535         # Run compiler with the configured stitching paddings + optional branding watermark
536         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
537         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
538                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
539         try:
540             # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
541             # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
542             # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
543             # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
544             # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
545             local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))
546             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
547             # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
548             # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
$4.3).
549             download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
550             return {"status": "success", "filepath": out_path, "download_url": download_url}
551         except Exception as e:
552             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
553
554     # --- CLI Debug Utility & Orchestration ---
555     def run_cli_diagnose_clip(video_path: str, timestamp: float):
556         """CLI debugging utility to examine segment properties around a timestamp."""
557         print("-" * 60)
558         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
559         print("-" * 60)
560
561         cfg = load_config() # legacy wrapper still works, returns dict for now
562
563         # 1. Validation check diagnostics
564         print("[DIAGNOSTIC] Running validation framework...")
565         results = registry.run_all(video_path, cfg)
566         for r in results:
567             status_symbol = "[PASS]" if r.passed else "[FAIL]"
568             print(f" {status_symbol} {r.validator_name}: {r.message}")
569             if r.details:
570                 print(f"      Details: {r.details}")
571
572         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
573         print("-" * 60)
574         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
575         import cv2
576         cap = cv2.VideoCapture(video_path)

```

```

577     if not cap.isOpened():
578         print("[FAIL] OpenCV could not open video.")
579         return
580
581     fps = cap.get(cv2.CAP_PROP_FPS)
582     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
583
584     start_frame = max(0, int((timestamp - 3.0) * fps))
585     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
586
587     # Measure frame-by-frame changes in sample region
588     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
589     prev_gray = None
590     motions = []
591
592     for _ in range(start_frame, end_frame):
593         ret, frame = cap.read()
594         if not ret:
595             break
596         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
597         gray = cv2.GaussianBlur(gray, (21, 21), 0)
598
599         if prev_gray is not None:
600             diff = cv2.absdiff(prev_gray, gray)
601             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
602             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
603             motions.append(motion_ratio)
604             prev_gray = gray
605
606     cap.release()
607
608     if motions:
609         avg_motion = sum(motions) / len(motions)
610         # Support both legacy dict (from wrapper) and future direct model
611         if hasattr(cfg, "indexing"):
612             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
613         else:
614             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
615         print(f" Sample Frame Count: {len(motions)}")
616         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
617         if avg_motion > threshold:
618             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
619         else:
620             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
621     else:
622         print(" [ERROR] No visual frames were extracted in the sample range.")
623
624     print("=" * 60)
625
626
627     def _enforce_startup_or_exit(config) -> None:
628         """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL
finding BEFORE
629         any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only
storage) OR, for
630         the exposed server (``include_exposure=True``, M9 hardening), a half-configured
exposure (edge
631         auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when
no
632         ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU

```

probe here ?

```
633         the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted so the
634         fail-fast wiring + the default-OFF no-op are unit-testable.""
635         try:
636             enforce_startup_checks(config, logger=logger, include_exposure=True)
637         except StartupCheckError as e:
638             logger.error("[STARTUP] refusing to start: %s", e)
639             sys.exit(1)
640
641
642     def main():
643         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
644         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
645         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
646         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
647         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
648         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
649         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
650         parser.add_argument("--host", default=None,
651                             help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0
when "
652                                     "security.edge_auth_enabled. Expose on a network ONLY
behind an auth gate.")
653         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
654
655         available_segmenters = list(segmenter_registry.list_available().keys())
656         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
657         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
658         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
659         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
660
661         args = parser.parse_args()
662
663         if args.setup:
664             # Invoke the diagnostics/bootstrap script (renamed from the old root
665             # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
666             # build system). Resolve its path from this file so `indexer --setup`
667             # works regardless of the caller's cwd.
668             import subprocess
669             doctor_path = os.path.join(
670                 os.path.dirname(os.path.abspath(__file__)),
671                 "scripts", "doctor.py",
672             )
673             print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
674             subprocess.run([sys.executable, doctor_path])
675             return
676
677         if args.trim_start is not None and args.trim_end is not None:
678             if not args.video_path:
679                 print("[ERROR] Please provide --video-path to extract a segment.")
680                 return
681
682         # Determine output path
```

```

683         out_filename = args.trim_output or "trimmed_segment.mp4"
684         if os.path.isabs(out_filename):
685             out_path = out_filename
686             out_dir = os.path.dirname(out_path)
687             out_name = os.path.basename(out_path)
688         else:
689             out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-
identical when unset)
690             out_path = os.path.join(out_dir, out_filename)
691             out_name = out_filename
692
693         os.makedirs(out_dir, exist_ok=True)
694         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
695
696         # global_config isn't initialized this early; load the typed config so the trim
697         # clip carries the same configured stitching paddings + watermark as a real
698         # highlight (watermark default-on).
699         stitching = load_app_config().stitching
700         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
701                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
702         try:
703             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
704             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
705         except Exception as e:
706             print(f"[ERROR] Failed to compile trimmed segment: {e}")
707         return
708
709     if args.debug_clip is not None:
710         if not args.video_path:
711             print("[ERROR] Please provide --video-path to debug a specific clip.")
712             return
713         run_cli_diagnose_clip(args.video_path, args.debug_clip)
714         return
715
716     # Initialize Global Config and DB
717     global global_config, db_instance
718     global_config = load_app_config() # returns typed AppConfig
719     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
profile
720
721     # The CLI --output-dir overrides the configured output directory. It now lives
722     # on the typed model (OutputConfig.output_dir), so every consumer ? including
723     # /api/compile ? reads the same value instead of a write-only runtime hack.
724     # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
725     # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
726     global_config.output.output_dir = resolve_output_dir(args.output_dir)
727     os.makedirs(global_config.output.output_dir, exist_ok=True)
728
729     db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
730     db_instance = Database(db_path)
731
732     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
733     # the executor is in-process. Mark them failed so pollers see a terminal state.
734     swept = db_instance.fail_stale_jobs()
735     if swept:
736         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
737
738     # CLI processing mode (no web UI needed)
739     if args.video_path and not args.server:

```



```

740         print(f"[CLI] Processing video file: {args.video_path}")
741         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
742         # The local-existence fast-fail only applies to a local input (stat the RESOLVED
743         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
744         # not a crash.
745         storage_cfg = getattr(global_config, "storage", None)
746         try:
747             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
748         except ValueError as e:
749             print(f"[FAIL] {e}")
750             return
751         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
752             print(f"[FAIL] Video file not found: {args.video_path}")
753             return
754         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
755         video_id = compute_video_id(local_video)
756         was_reingest = db_instance.get_video(video_id) is not None
757
758         # Validate (with-context variant surfaces ffprobe_meta for the report)
759         print("[CLI] Validating...")
760         val_results, val_context = registry.run_all_with_context(local_video,
761         global_config)
762         failures = [r for r in val_results if not r.passed]
763
764         # Save video status
765         status = "failed" if failures else "processed"
766         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
767         in val_results])
768
769         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
770         "default_segmenter", "motion")
771         segmenter_actual = None
772         segmentation_ran = False
773         try:
774             print("\n" + "="*40)
775             print("VALIDATION SUMMARY")
776             print("="*40)
777             for r in val_results:
778                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
779                 print(f"{status_symbol} {r.validator_name}: {r.message}")
780             print("="*40 + "\n")
781
782             if failures:
783                 print("[FAIL] Video failed checks. Indexing halted.")
784                 return
785
786             # Index
787             segmenter_cls = segmenter_registry.get(seg_name)
788             if not segmenter_cls:
789                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
790                 return
791
792             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
793             segmenter_actual = seg_name
794             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
795             segmenter = segmenter_cls(db_instance, global_config)
796             result = segmenter.process_video(video_id, local_video,
797             max_frames=args.max_frames)
798             segmentation_ran = True
799
800             if isinstance(result, Failure):
801                 print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")

```

```

798         print("[FAIL] Indexing halted for video.")
799         segments = []
800     else:
801         segments = result.value if hasattr(result, "value") else result
802         print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
803         # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
804         _finalize_reingest(video_id, segments, play_wm, cand_wm)
805     finally:
806         # Always emit the per-video observability report. Never raises.
807         paths = emit_indexer_report(
808             db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
809             val_results=val_results, val_context=val_context,
810             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
811             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
812         )
813     if paths:
814         print(f"[REPORT] Technical report : {paths.get('technical_md')}")
815         print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
816         print(f"[REPORT] Machine JSON      : {paths.get('json')}")
817     return
818
819     # Start Local Server Mode
820     if args.server or not args.video_path:
821         bind_host = _resolve_bind_host(global_config, args.host) # P0c: config/profile-
driven (default 127.0.0.1)
822         print(f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{args.port}...")
823         if bind_host not in ("127.0.0.1", "localhost", ":::1"):
824             print(f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
825                 f"gate (edge_auth_enabled / a reverse proxy) is in front, and set "
826                 f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header guard
defaults to "
827                 f"loopback-only and will 400 forwarded requests otherwise).")
828         # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
829         # in-process wake-timer for any parked 'waiting' job died with the old process,
so
830         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
831         _get_executor().submit(_drain_due_jobs)
832         uvicorn.run(app, host=bind_host, port=args.port)
833
834     if __name__ == "__main__":
835         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
836         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
837         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
838         # `backend.main.db_instance` = None; the async-job drain then crashes silently on
839         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
840         # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
841         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
842         import backend.main as _bm
843
844         _bm.main()
845

```

Now let me check the SecurityConfig and verify Starlette's TrustedHostMiddleware port-stripping behavior.

7. user (2026-06-21T15:27:18.195Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
index 03a4498..d77f98e 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -67,18 +67,28 @@ class WasbIndexConfig(BaseModel):
     wsl_stage_dir: str = "~/clips"
     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
     velocity_threshold: float = 0.02
+
+ # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner. ---
+ # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
+ # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4 CPU
+ fallback
+ # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-CPU
+ downgrade).
+ device: str = "cuda" # auto | cuda | mps | cpu
+ python_bin: str = "python" # the `wasb` conda env python (invoked by path, no `conda
+ activate`)
+ # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
+ # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates, regenerable
+ # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
+ resumes.
+ keep_frames: bool = False
+ # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
+ disabled.
+ min_free_gb: float = 0.0
+ # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
+ # on the fly and feed frames straight to the detector, skipping the slow per-frame PNG
+ # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
+ # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is lossless),
+ # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms it.
+ stream_video: bool = False
+ # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
+ # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs for a
+ # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical to the
+ # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream == WSL
+ disk,
+ # 1194/1195 frames byte-identical; native run-to-run byte-identical / deterministic).
+ # Tri-state:
+ # None = per-runner default ? the WSL runner keeps DISK extraction (unchanged
+ Windows
+ # behaviour); the Linux-native runner (GPU box) defaults to STREAMING (the
+ perf win).
+ # True/False = force on/off for BOTH runners (e.g. set False for a container cv2 can't
+ seek).
+ stream_video: Optional[bool] = None

class FusionConfig(BaseModel):
@@ -117,6 +127,12 @@ class FusionConfig(BaseModel):
class IndexingConfig(BaseModel):
    default_segmenter: str = "yolo_hybrid"
    use_gpu: bool = True
+
+ # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
+ # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
+ # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
+ # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
+ # RALLY_DETECTOR_IMPL env var overrides this. See
+ pipeline/detectors/{stub,native_wasb}_runner.py.
+ detector_impl: str = "auto"
+ motion_threshold: float = 0.08
```

```

    min_segment_duration: float = 3.0
    dead_time_merge_gap: float = 2.0
@@ -129,14 +145,44 @@ class IndexingConfig(BaseModel):
    yolo_tracker: str = "bytetrack.yaml"
    yolo_prefetch_workers: int = 2
    min_action_window_duration: float = 2.0
-   # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF (default ?
-   # behaviour unchanged). When > 0, a candidate window longer than this many seconds is
-   # recursively split at its largest internal inactivity gap (? ``window_min_split_gap`` s ?
-   # the most likely rally boundary), so one window can't swallow several rallies + the dead
-   # time between them. Never merges, only cuts (recall can't drop). Measured recommended
value
-   # 12.0 (golden n=6: ?F1 +0.139, 6/0 sign-test, merge_rate 0.523?0.038); kept OFF until
promoted.
-   max_window_duration: float = 0.0
+   # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0 = a
window
+   # longer than this many seconds is recursively split at its largest internal inactivity gap
(?)
+   # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't swallow
+   # several rallies + the dead time between them. Never merges, only cuts (recall can't
drop).
+   # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip. Measured
n=13
+   # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0, p=0.0005;
cap=6
+   # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate 0.651?0.161).
***
+   max_window_duration: float = 6.0
    window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split point
+   # HARD-CAP fallback for W1: when True, an over-long window with NO internal inactivity gap
+   # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob) is
+   # chopped at its midpoint until ? max_window_duration, making the cap a true upper bound.
+   # Only cuts (recall can't drop). OFF by default until measured/promoted.
+   window_hard_cap: bool = False
+   # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving slowly
+   # (picked up / walked) above velocity_thresh, so the window over-extends its END (the
measured
+   # [?end| gap vs golden). Trim the post-rally tail back to the last frame whose trailing

```

8. user (2026-06-21T15:27:19.668Z)

```

277-   verbose_http: bool = False
278-
279-
280:class SecurityConfig(BaseModel):
281-   # When set, /api/process video_path must resolve under this directory (hosted/tenant
282-   # mode). Default None preserves the single-user local 'any local file' workflow.
283-   allowed_video_root: Optional[str] = None
284-
285-   # --- Chunked upload (B2, PR-2) ---
286-   # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287-   # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288-   # contain upload_dir or /api/process will reject the uploaded paths.
289-   upload_dir: str = "output/uploads"
290-   # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
--
294-   allowed_upload_extensions: List[str] = ["mp4", "mov"]
295-
296-   # --- M9 edge auth (Cloudflare Access, D9) ---
297-   # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local single-
user
298-   # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-Jwt-
Assertion`
299-   # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the

```

```

identity
300- # header) and tags the job with the verified user_id (owner_id). The team domain (e.g.
301- # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is the CF
Access
302- # application's audience tag. NO secrets here ? these are public identifiers.
303: edge_auth_enabled: bool = False
304- cf_team_domain: str = "" # e.g. https://<team>.cloudflareaccess.com (issuer +
/certs JWKS)
305- cf_access_aud: str = "" # the CF Access application AUD tag the token must carry
306- # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at import
in

```

9. assistant (2026-06-21T15:27:22.869Z)

I have confirmed `edge\_auth\_enabled: bool = False` defaults. Now let me verify the critical Starlette behavior claims: (1) does TrustedHostMiddleware strip the port before matching, and (2) does it short-circuit before CORS. Let me check the installed Starlette source.

10. user (2026-06-21T15:27:24.688Z)

1.0.0

C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\starlette

11. user (2026-06-21T15:27:27.133Z)

```

1     from __future__ import annotations
2
3     from collections.abc import Sequence
4
5     from starlette.datastructures import URL, Headers
6     from starlette.responses import PlainTextResponse, RedirectResponse, Response
7     from starlette.types import ASGIApp, Receive, Scope, Send
8
9     ENFORCE_DOMAIN_WILDCARD = "Domain wildcard patterns must be like '*.example.com'."
10
11
12     class TrustedHostMiddleware:
13         def __init__(
14             self,
15             app: ASGIApp,
16             allowed_hosts: Sequence[str] | None = None,
17             www_redirect: bool = True,
18         ) -> None:
19             if allowed_hosts is None:
20                 allowed_hosts = [""]
21
22             for pattern in allowed_hosts:
23                 assert "" not in pattern[1:], ENFORCE_DOMAIN_WILDCARD
24                 if pattern.startswith("") and pattern != "":
25                     assert pattern.startswith("*."), ENFORCE_DOMAIN_WILDCARD
26             self.app = app
27             self.allowed_hosts = list(allowed_hosts)
28             self.allow_any = "" in allowed_hosts
29             self.www_redirect = www_redirect
30
31         async def __call__(self, scope: Scope, receive: Receive, send: Send) -> None:
32             if self.allow_any or scope["type"] not in (
33                 "http",
34                 "websocket",
35             ): # pragma: no cover
36                 await self.app(scope, receive, send)
37                 return
38
39             headers = Headers(scope=scope)

```

```

40         host = headers.get("host", "").split(":")[0]
41         is_valid_host = False
42         found_www_redirect = False
43         for pattern in self.allowed_hosts:
44             if host == pattern or (pattern.startswith("**") and
host.endswith(pattern[1:])):
45                 is_valid_host = True
46                 break
47             elif "www." + host == pattern:
48                 found_www_redirect = True
49
50         if is_valid_host:
51             await self.app(scope, receive, send)
52         else:
53             response: Response
54             if found_www_redirect and self.www_redirect:
55                 url = URL(scope=scope)
56                 redirect_url = url.replace(netloc="www." + url.netloc)
57                 response = RedirectResponse(url=str(redirect_url))
58             else:
59                 response = PlainTextResponse("Invalid host header", status_code=400)
60             await response(scope, receive, send)
61

```

12. assistant (2026-06-21T15:27:34.246Z)

Key confirmations from Starlette 1.0.0:

- Line 40: ``host = headers.get("host", "").split(":")[0]`` ? port IS stripped before matching. So ``127.0.0.1:8000`` matches ``127.0.0.1``. Good.
- BUT: IPv6 ``[:,1]:8000`` ? ``split(":")[0]`` yields ``["`` ? NOT ``":1"`` or ``[:,1]``. So IPv6 loopback Host header would 400.
- Line 32-37: ``allow_any`` (``*``) short-circuits entirely.

Now the critical middleware-order question. In Starlette/FastAPI, ``add_middleware`` prepends, so the LAST-added middleware is OUTERMOST. CORS is added first (line 70), TrustedHost second (line 96), so TrustedHost is outermost ? it runs BEFORE CORS. Let me verify how FastAPI/Starlette builds the middleware stack to confirm ordering and the CORS-preflight interaction.

13. user (2026-06-21T15:27:43.239Z)

```

0 TrustedHostMiddleware
1 CORSMiddleware

```

14. assistant (2026-06-21T15:27:56.445Z)

Let me verify how Starlette assembles these into the stack (which one is outermost) and confirm the practical request behavior with the TestClient.

15. user (2026-06-21T15:27:57.948Z)

```

def build_middleware_stack(self) -> ASGIApp:
    debug = self.debug
    error_handler = None
    exception_handlers: dict[Any, ExceptionHandler] = {}

    for key, value in self.exception_handlers.items():
        if key in (500, Exception):
            error_handler = value
        else:
            exception_handlers[key] = value

    middleware = (
        [Middleware(ServerErrorMiddleware, handler=error_handler, debug=debug)]
        + self.user_middleware

```

```

        + [Middleware(ExceptionMiddleware, handlers=exception_handlers, debug=debug)]
    )

    app = self.router
    for cls, args, kwargs in reversed(middleware):
        app = cls(app, *args, **kwargs)
    return app

```

16. assistant (2026-06-21T15:28:02.413Z)

Confirmed the ordering. The list is `[ServerError, TrustedHost, CORS, Exception, router]`, built inner-first via `reversed`, so the wrapping order outermost?innermost is: ServerError ? TrustedHost ? CORS ? Exception ? router. So ****TrustedHost runs BEFORE CORS**** ? a disallowed Host is rejected before CORS ever sees the request, so no CORS headers leak. Good.

Now let me empirically verify the full behavior with the TestClient: the suite passes, a CORS preflight from a disallowed host, IPV6 loopback, and a wildcard `` bypass.

17. user (2026-06-21T15:28:14.972Z)

```

20:58:14 [INFO] httpx: HTTP Request: GET http://testserver/api/health "HTTP/1.1 200 OK"
20:58:14 [INFO] httpx: HTTP Request: GET http://testserver/api/health "HTTP/1.1 200 OK"
20:58:14 [INFO] httpx: HTTP Request: GET http://testserver/api/health "HTTP/1.1 400 Bad Request"
20:58:14 [INFO] httpx: HTTP Request: GET http://testserver/ "HTTP/1.1 200 OK"
20:58:14 [INFO] httpx: HTTP Request: OPTIONS http://testserver/api/process "HTTP/1.1 400 Bad Request"
20:58:14 [INFO] httpx: HTTP Request: OPTIONS http://testserver/api/process "HTTP/1.1 200 OK"
20:58:14 [INFO] httpx: HTTP Request: GET http://testserver/api/health "HTTP/1.1 400 Bad Request"
20:58:14 [INFO] httpx: HTTP Request: GET http://testserver/api/health "HTTP/1.1 400 Bad Request"
20:58:14 [INFO] httpx: HTTP Request: GET http://testserver/api/health "HTTP/1.1 400 Bad Request"
health default          200
health 127.0.0.1:8000    200
health evil.com          400
root /                  200
preflight bad-host      400 | ACAO leaked: False
preflight good-host     200 | ACAO: *
health host='[:1]:8000'  400
health host='[:1]'       400
health host=':1'         400

```

18. assistant (2026-06-21T15:28:21.380Z)

Empirical results confirm:

- testserver default: 200 (suite passes)
- loopback with port: 200 (port stripped correctly)
- foreign host: 400
- static root ``: 200
- preflight from bad host: 400, ****no CORS headers leaked**** (ACAO absent) ? TrustedHost outermost works correctly
- preflight from good host: 200 with ACAO
- ****IPV6 `[:1]:8000`, `[:1]`, `:1` all 400**** ? IPV6 loopback Host headers are rejected despite ``:1`` and ``[:1]`` being in the allowlist, because Starlette's `.split(":")[0]` mangles them.

Now let me run the actual new test file and confirm byte-identity of the default path against master. Let me check the git diff of main.py to see exactly what changed.

19. user (2026-06-21T15:28:21.692Z)

<persisted-output>

Output too large (37.7KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\tool-results\b8nlydyhr.txt

```

Preview (first 2KB):
diff --git a/backend/main.py b/backend/main.py
index 0f48c7e..9eea4e9 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -2,14 +2,18 @@ import os
import sys
import argparse
import logging
-import threading
-import traceback
import uuid
-from concurrent.futures import ThreadPoolExecutor
+from urllib.parse import quote
import uvicorn
from dotenv import load_dotenv

-load_dotenv()
+# PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the packaged
+# launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None) keeps the
+# exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare load_dotenv().
+# stdlib-only import ? keeps this pre-pydantic line light.
+from backend.utils.app_home import env_file_to_load, resolve_output_dir
+
+load_dotenv(env_file_to_load())

# Centralized logging setup for the indexer (hot paths use logger, CLI output uses print for
direct user-facing messages).
logging.basicConfig(
@@ -20,6 +24,7 @@ logging.basicConfig(
    logger = logging.getLogger(__name__)
    from fastapi import FastAPI, HTTPException, Request
    from fastapi.middleware.cors import CORSMiddleware
+from starlette.middleware.trustedhost import TrustedHostMiddleware
    from pydantic import BaseModel
    from typing import List, Dict, Any, Optional

@@ -31,30 +36,101 @@ from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.compiler import VideoCompiler
    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
    from backend.utils.paths import resolve_under_root, safe_output_filename, validate_input_path
-from backend.config import load_config as load_app_config, AppConfig, StitchingConfig # new
typed config
+from backend.utils.redact import redact_secrets # P0b: mask secret-shaped fields in
/api/config
+from backend.config import ( # new typed config
+    load_config as load_app_config,
+    AppConfig,
+    StitchingConfig,
+    ...
</persisted-output>

```

20. assistant (2026-06-21T15:28:26.056Z)

The diff against master is large because master is behind several merged PRs (P0a, P0b, M9 etc.). Let me compare against the worktree's actual base. The branch is `feat/packaging-p0c-bind-host` and is "up to date with origin/master" per status, but the recent commits show P0a/P0b are already in. Let me check what the actual P0c change set is by diffing against HEAD (uncommitted changes only), since the change is uncommitted (modified main.py + untracked test).

21. user (2026-06-21T15:28:27.767Z)

```

=== HEAD is ===
fc78fce feat(pkg): P0b ? redact secret-shaped fields from the unauthenticated GET /api/config

```



```

(#295)
=== diff HEAD (the actual P0c change) ===
diff --git a/backend/main.py b/backend/main.py
index 4d9e487..9eea4e9 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -24,6 +24,7 @@ logging.basicConfig(
    logger = logging.getLogger(__name__)
    from fastapi import FastAPI, HTTPException, Request
    from fastapi.middleware.cors import CORSMiddleware
+from starlette.middleware.trustedhost import TrustedHostMiddleware
    from pydantic import BaseModel
    from typing import List, Dict, Any, Optional

@@ -74,6 +75,46 @@ app.add_middleware(
    allow_headers=["*"],
)

+
+def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
+    """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
+    malicious
+    web page can point a domain it controls at 127.0.0.1 and script requests to the local
+    server;
+    the browser sends ``Host: attacker.com``, which this rejects (the server only answers
+    loopback
+    Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the default
+    keeps
+    Starlette's TestClient host ``testserver`` so the in-process suite isn't broken. Read from
+    the
+    ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a
+    routable/edge
+    deploy sets its public host (or ``*`` when fronted by an auth gate that validates the
+    host)."""
+    src = os.environ if env is None else env
+    raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
+    hosts = [h.strip() for h in raw.split(",") if h.strip()]
+    return hosts or ["localhost", "127.0.0.1", ":", ":", ":", "testserver"]
+
+
+## DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are
+honoured
+## (a remote page can't read responses cross-origin AND its Host header is rejected here).
+www_redirect
+## off so an unexpected ``www`` host is rejected rather than 307-redIRECTED.
+app.add_middleware(
+    TrustedHostMiddleware,
+    allowed_hosts=_allowed_hosts_from_env(),
+    www_redirect=False,
+)
+
+
+def _resolve_bind_host(config: Optional[AppConfig] = None, cli_host: Optional[str] = None) ->
+str:
+    """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
+    ``0.0.0.0``
+    when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS an auth
+    gate
+    in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a routable
+    interface with no auth gate is unsafe, hence the localhost default; widen it
+    consciously."""
+    if cli_host and cli_host.strip():
+        return cli_host.strip()
+    env = os.environ.get("RALLY_BIND_HOST")
+    if env and env.strip():

```

```

+         return env.strip()
+     sec = getattr(config, "security", None)
+     if getattr(sec, "edge_auth_enabled", False):
+         return "0.0.0.0" # behind a Cloudflare-Access-style edge gate (M9) ? bind all
interfaces
+     return "127.0.0.1"
+
+ # Serves static frontend files directly (now under api/ per approved structure).
+ # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from an
+ # arbitrary directory finds the shipped assets instead of creating a stray
+ ./backend/api/static.
@@ -606,6 +647,9 @@ def main():
    parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a video
clip to run detailed diagnostics on")
    parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
    parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI server
on")
+   parser.add_argument("--host", default=None,
+       help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0 when "
+       "security.edge_auth_enabled. Expose on a network ONLY behind an
auth gate.")
    parser.add_argument("--max-frames", type=int, help="Limit processing to the first N sub-
sampled frames for debugging")

    available_segmenters = list(segmenter_registry.list_available().keys())
@@ -774,12 +818,18 @@ def main():

    # Start Local Server Mode
    if args.server or not args.video_path:
-       print(f"[SERVER] Starting local FastAPI server on http://localhost:{args.port}...")
+       bind_host = _resolve_bind_host(global_config, args.host) # P0c: config/profile-driven
(default 127.0.0.1)
+       print(f"[SERVER] Starting local FastAPI server on http://{bind_host}:{args.port}...")
+       if bind_host not in ("127.0.0.1", "localhost", "::1"):
+           print(f"[SERVER] ? Binding {bind_host} exposes the API on the network ? ensure an
auth "
+               f"gate (edge_auth_enabled / a reverse proxy) is in front, and set "
+               f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header guard defaults
to "
+               f"loopback-only and will 400 forwarded requests otherwise).")
        # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3): the
        # in-process wake-timer for any parked 'waiting' job died with the old process, so
        # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
        _get_executor().submit(_drain_due_jobs)
-       uvicorn.run(app, host="127.0.0.1", port=args.port)
+       uvicorn.run(app, host=bind_host, port=args.port)

    if __name__ == "__main__":
        # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads` resolve
the

```

22. assistant (2026-06-21T15:28:33.425Z)

Now I have the precise, isolated P0c change. Let me note a byte-identity observation: the default server-start print changed from `http://localhost:{port}` to `http://{bind\_host}:{port}` where `bind\_host` resolves to `127.0.0.1` by default. So the **printed string changed from "localhost" to "127.0.0.1" in the default path. That's a user-facing string change, not a behavior change (`uvicorn.run` previously hardcoded `127.0.0.1` anyway, so the actual bind is byte-identical). Worth flagging as a nit since the contract says "byte-identical."

Now let me run the new test file plus a quick check on the broader TestClient suite to confirm nothing breaks.

23. user (2026-06-21T15:28:42.156Z)

..... [100%]
10 passed in 5.43s

24. user (2026-06-21T15:29:24.039Z)

..... [29%]
..... [58%]
..... [87%]
..... [100%]
247 passed, 1030 deselected in 34.96s

25. assistant (2026-06-21T15:29:30.989Z)

All 247 API/TestClient tests pass ? the existing suite (using `testserver` Host) is not broken. The hard contract holds for the default path.

Now let me verify the edge-auth footgun concern: when `edge\_auth\_enabled=True`, bind becomes `0.0.0.0` but the allowlist was resolved at import-time from env only. Let me confirm the warning logic gap precisely. The warning at lines 823-827 only fires when `bind\_host` not in ("127.0.0.1", "localhost", ":::1"). When `edge\_auth\_enabled=True` and no `--host`/env, `bind\_host` = `"0.0.0.0"`, which is NOT in that tuple, so the warning DOES fire. Good ? the footgun is surfaced at bind time.

Let me double-check one edge case in the warning tuple: it lists `:::1` but not `[:::1]`, and if someone sets `RALLY\_BIND\_HOST=localhost` the warning is correctly suppressed. That's consistent. Let me also confirm `\_enforce\_startup\_or\_exit` / `\_enforce\_startup\_checks` would catch the half-configured exposure (edge auth on without allowlist) to see if the footgun is mitigated upstream.

26. user (2026-06-21T15:29:32.100Z)

```
backend/config/models.py:297: # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is
required/verified (the local single-user
backend/config/models.py:303: edge_auth_enabled: bool = False
backend/config/startup.py:18:**Exposure-safety posture (server only,
`include_exposure=True``). ** When the engine is *exposed*
backend/config/startup.py:19:behind an edge gate (the owner flips ``security.edge_auth_enabled``
on for the Cloudflare-Access
backend/config/startup.py:26:``include_exposure=True``; the worker reads ``--video-uri`` (not
untrusted tenant paths), so it is
backend/config/startup.py:30:a strict **no-op** ? exactly the pre-M5 behaviour; with
``edge_auth_enabled=False`` (today's default)
backend/config/startup.py:91: """Exposure-safety posture for an *exposed* HTTP server. Gated
on ``security.edge_auth_enabled``
backend/config/startup.py:97: if not sec or not getattr(sec, "edge_auth_enabled", False):
backend/config/startup.py:109: "security.edge_auth_enabled is ON (engine exposed) but
security.allowed_video_root is "
backend/config/startup.py:123: f"security.edge_auth_enabled is ON but {missing} is
not configured ? the Cf-Access JWT "
backend/config/startup.py:135: "security.edge_auth_enabled is ON but PyJWT[crypto] is
not importable ? Cf-Access "
backend/config/startup.py:147: include_exposure: bool = False,
backend/config/startup.py:153: ``include_exposure`` adds the exposure-safety posture
(``_exposure_checks``) ? the HTTP **server**
backend/config/startup.py:155: DEFAULT-OFF: no profile ? no profile checks;
``edge_auth_enabled=False`` ? no exposure checks.
backend/config/startup.py:162: # runs with no profile set), gated on edge_auth_enabled
(default-OFF). Server-only.
backend/config/startup.py:163: if include_exposure:
backend/config/startup.py:247: include_exposure: bool = False,
backend/config/startup.py:253: ``include_exposure`` (server-only) adds the exposure-safety
posture. Returns the full findings
backend/config/startup.py:258: config, cuda_available=cuda_available, env=env,
include_exposure=include_exposure)
```

27. user (2026-06-21T15:29:34.901Z)

```
91      """Exposure-safety posture for an *exposed* HTTP server. Gated on
92      ``security.edge_auth_enabled``
93      (default False ? ``[]`` = strict no-op, today's behaviour). When edge auth is ON ?
94      the "I am exposing this engine behind the CF-Access gate" signal ? the gate alone isn't
95      enough: verify the
96      rest of the posture at boot so a half-configured exposure fails fast instead of
97      leaking / 500-ing
98      per request. Config-only (no IO beyond the ``find_spec`` auth-extra probe)."""
99      sec = getattr(config, "security", None)
100      if not sec or not getattr(sec, "edge_auth_enabled", False):
101          return [] # DEFAULT-OFF: edge auth off ? no posture enforcement (byte-identical
102      to today).
103      out: list[StartupCheck] = []
104      # (1) The path-jail is load-bearing once authenticated tenants can hit /api/process:
105      WITHOUT it,
106      # a tenant can point the engine at an ARBITRARY local file (allowed_video_root unset
107      = "any local
108      # file", the single-user default). Fatal for an exposed engine.
109      root = getattr(sec, "allowed_video_root", None)
110      if not root or not str(root).strip():
111          out.append(StartupCheck(
112              LEVEL_ERROR, "EXPOSED_NO_VIDEO_ROOT",
113              "security.edge_auth_enabled is ON (engine exposed) but
114              security.allowed_video_root is "
115              "unset ? an authenticated tenant could make /api/process read an arbitrary
116              local file. "
117              "Set allowed_video_root to a jail directory before exposing the engine.",
118              ))
119      # (2) Edge auth cannot verify a token without the team domain + AUD. Today that only
120      fails on the
121      # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate never
122      starts
123      # "healthy" then 401s every request.
124      team = (getattr(sec, "cf_team_domain", "") or "").strip()
125      aud = (getattr(sec, "cf_access_aud", "") or "").strip()
126      if not team or not aud:
127          missing = " + ".join(n for n, v in (("cf_team_domain", team), ("cf_access_aud",
128          aud)) if not v)
129          out.append(StartupCheck(
130              LEVEL_ERROR, "EXPOSED_EDGE_AUTH_INCOMPLETE",
131              f"security.edge_auth_enabled is ON but {missing} is not configured ? the Cf-
132              Access JWT "
133              "cannot be verified (every request would fail). Set security.cf_team_domain
134              "
135              "(e.g. https://<team>.cloudflareaccess.com) and security.cf_access_aud (the
136              CF Access "
137              "application AUD tag).",
138              ))
139      # (3) The auth extra (PyJWT[crypto]) is lazy-imported by
140      auth.verify_cf_access_token. If it's
141      # missing, every request 500s at verify time ? a boot WARN beats a per-request 500.
142      Warning, not
143      # fatal: the auth module's own ImportError is the final authority.
144      if not _auth_extra_available():
145          out.append(StartupCheck(
146              LEVEL_WARN, "EXPOSED_AUTH_EXTRA_MISSING",
147              "security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ? Cf-
148              Access "
```

```

136             "verification will fail at request time. Install the 'auth' extra (pip
install -e .[auth]).",
137         ))
138
139     return out
140
141
142     def run_startup_checks(
143         config: Any,
144         *,
145         cuda_available: Optional[bool] = None,
146         env: Optional[Mapping[str, str]] = None,
147         include_exposure: bool = False,
148     ) -> list[StartupCheck]:
149         """Return the startup findings for ``config``.
150

```

28. assistant (2026-06-21T15:29:43.675Z)

This is the key finding for concern #2. The exposure-safety startup checks (run via ``_enforce_startup_or_exit` ? `_enforce_startup_checks(..., include_exposure=True)``) verify ``allowed_video_root``, ``cf_team_domain``, ``cf_access_aud``, and the auth extra when ``edge_auth_enabled=True`` ? but they do **NOT** check that ``RALLY_ALLOWED_HOSTS`` is set. So with edge auth ON, bind = ``0.0.0.0``, and ``RALLY_ALLOWED_HOSTS`` unset, the engine **starts successfully** (passes all fatal checks) but the import-time allowlist defaults to loopback-only ? meaning every forwarded public-Host request from Cloudflare Access gets a 400.

The only mitigation is the bind-time ``print()`` warning (lines 824-827), which does fire for ``0.0.0.0``. So the footgun IS surfaced, but only as a stderr print ? not enforced as a fatal startup check the way the parallel ``allowed_video_root`` exposure gap is. This is an inconsistency: the exposure posture treats a missing path-jail as FATAL but a missing host-allowlist (which would 400 100% of real traffic) as only a soft print. Let me verify the warning actually reaches the operator and confirm there's no env-signal coupling for the allowlist.

Let me confirm the docs/PACKAGING_PLAN.md surfaces this, to assess whether it's "adequately documented."

29. user (2026-06-21T15:29:46.500Z)

30. user (2026-06-21T15:29:52.458Z)

```

--- env ref count in code ---
backend/main.py:83: Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated)
overrides; the default keeps
backend/main.py:88: raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
backend/main.py:826: f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-
header guard defaults to "
Binary file backend/__pycache__/main.cpython-313.pyc matches

```

31. assistant (2026-06-21T15:30:23.263Z)

``PACKAGING_PLAN.md`` doesn't exist in the worktree (it's a planning doc referenced in the prompt but not in-repo), and ``RALLY_ALLOWED_HOSTS`` is documented only in the docstring + the bind-time print. There's no standalone operator doc. The footgun surfacing rests entirely on the runtime print warning.

I have everything I need. Let me do a final precise check on the ``getattr`` null-safety in ``_resolve_bind_host`` (concern #4) ? confirm ``_resolve_bind_host(None, None)`` works (already tested, returns ``127.0.0.1``) and that ``getattr(sec, "edge_auth_enabled", False)`` when ``sec`` is ``None`` returns ``False`` safely.

``getattr(None, "edge_auth_enabled", False)`` ? ``False`` (safe; the test

``test_bind_no_config_is_localhost`` proves it). ``getattr(config, "security", None)`` when ``config=None`` ? ``None``. Both chains are null-safe.

Let me compile the findings.

Review findings

****[minor]** backend/main.py:90 (``_allowed_hosts_from_env``) + Starlette trustedhost.py:40 ? IPv6 loopback Host headers are silently rejected. The default allowlist includes ``:::1`` and ``:::1``, but Starlette's ``TrustedHostMiddleware`` matches with ``host = headers.get("host", "").split(":")[0]``. A real IPv6 Host header is ``:::1:8000`` ? ``split(":")[0]`` yields ``:::1`` (and bare ``:::1`` ? ``""``), neither of which matches any allowlist entry. Empirically verified: ``:::1:8000``, ``:::1``, and ``:::1`` all return ``400``. So if a user binds/visits the appliance over IPv6 loopback, every request is wrongly rejected. The ``:::1``/``:::1`` entries in the default are dead ? they can never match given Starlette's port-stripping. Fix: document that IPv6 loopback is unsupported by this guard, or (cleaner) front it with a host-normalizer. The prompt flagged this as minor and it is ? IPv4 loopback (the actual default bind ``127.0.0.1``) works fine.

****[minor]** backend/config/startup.py:91-139 vs backend/main.py:96-100 ? the edge-auth exposure footgun is surfaced only as a soft print, not enforced like its sibling checks. When ``edge_auth_enabled=True``, ``_resolve_bind_host`` returns ``0.0.0.0`` (main.py:114-115) but the ``TrustedHost`` allowlist is frozen at import from ``RALLY_ALLOWED_HOSTS`` only (main.py:98) ? it cannot see config. If ``RALLY_ALLOWED_HOSTS`` is unset, the loopback-only default 400s ``100%`` of forwarded (public-Host) requests from the edge gate. The ``_exposure_checks`` startup gate already FATALs on a missing ``allowed_video_root`` and missing ``cf_team_domain``/``cf_access_aud``, but does **not** check the host allowlist ? so a half-configured exposure (host-allowlist unset) starts "healthy" then 400s every real request. The only mitigation is the bind-time ``print()`` warning (main.py:823-827), which does correctly fire for ``0.0.0.0`` (verified: ``0.0.0.0`` is not in the suppress-tuple). Fix: add an ``include_exposure`` startup check that warns/errors when ``edge_auth_enabled`` and ``RALLY_ALLOWED_HOSTS`` is unset/loopback-only, mirroring the existing ``EXPOSED_`` checks ? that's the cleaner option than the env-only docstring note. Not a blocker because the default path is unaffected and the print does surface it.

****[nit]** backend/main.py:822 ? default server-start banner string changed from ``localhost`` to the resolved bind host. Was ``http://localhost:{port}``; now ``http://{bind_host}:{port}`` which prints ``http://127.0.0.1:8000`` on the default path. The ``bind`` is byte-identical (``uvicorn.run`` got ``127.0.0.1`` before too), but the user-visible banner text changed ``localhost`` ? ``127.0.0.1``. Purely cosmetic; flagging only because the contract says "byte-identical." No functional impact.

Items checked and CLEARED (no issue)

- ****Concern 1 ? legitimate access:**** ``testserver`` is allowlisted (main.py:90) ? full ``TestClient`` suite passes (247 API tests + 10 new = green). Loopback Host **with port** ``127.0.0.1:8000`` ? ``200`` (Starlette strips the port at trustedhost.py:40). Health, static ``/``, and a good-host CORS preflight all return 200. No legitimate IPv4 path is wrongly 400'd.
- ****Concern 3 ? middleware order & CORS:**** Verified via ``build_middleware_stack`` that the wrap order is ``ServerError`` ? **TrustedHost** ? CORS ? router, i.e. ``TrustedHost`` is **outermost** and runs **before** CORS (FastAPI prepends; the stack is built inner-first via ``reversed``). Empirically, a preflight ``OPTIONS`` from a disallowed Host returns 400 with **no** ``Access-Control-Allow-Origin`` header leaked. ``www_redirect=False`` is correct ? it prevents a 307 to a ``www.`` host (an open-redirect-ish surprise) and just 400s instead.
- ****Concern 2 (precedence) ? `\_resolve\_bind\_host` order** cli > env > profile > 127.0.0.1: correct and safe; matches the four tests (``test_bind_cli_host_wins``, ``test_bind_env_beats_profile``, ``test_bind_edge_auth_profile_widens_to_all_interfaces``, ``test_bind_defaults_to_localhost``).
- ****Concern 4 ? null-safety:**** ``getattr(config, "security", None)`` then ``getattr(sec, "edge_auth_enabled", False)`` is null-safe for ``config=None`` AND ``security=None`` ? returns ``127.0.0.1`` (proven by ``test_bind_no_config_is_localhost``). The ``--host`` default ``None`` flows through ``if cli_host and cli_host.strip()`` correctly.
- ****Hard contract (byte-identity of default path):**** Holds. Default bind = ``127.0.0.1`` (unchanged), default allowlist preserves ``testserver``, import does zero filesystem I/O (env-only), and the full existing ``TestClient`` suite is green. The only default-path delta is the cosmetic banner string above.

****Verdict:**** Not a blocker ? the hard contract holds and the default path is behaviorally byte-identical; ship-able, but consider promoting the edge-auth host-allowlist footgun (finding 2) to a startup check for parity with the existing `EXPOSED\_\*` gates, and document the IPv6-loopback limitation (finding 1).